

CSDS 2026 – HW 2

Remark: Using AI to refine plots and RMarkdown formatting, and to improve the grammar of written answers.

Question 1

(a) Create and visualize a new “Distribution 2”:

- A combined dataset (n=800) that is negatively skewed (tail stretches to the left).
- Change the mean and standard deviation of d1, d2, and d3 to achieve this new distribution.
- Compute the mean and median, and draw lines showing the mean (solid) and median (dashed).

```
d1 <- rnorm(n = 500, mean = 40, sd = 6)
d2 <- rnorm(n = 200, mean = 22.5, sd = 6)
d3 <- rnorm(n = 100, mean = 5, sd = 6)
d123 <- c(d1, d2, d3)
```

```
mean_d123 <- mean(d123)
mean_d123
```

```
## [1] 31.33414
```

```
median_d123 <- median(d123)
median_d123
```

```
## [1] 35.50334
```

```
library(ggplot2)
```

```
# --- predefine format for the plot ---
theme_set(theme_minimal())
```

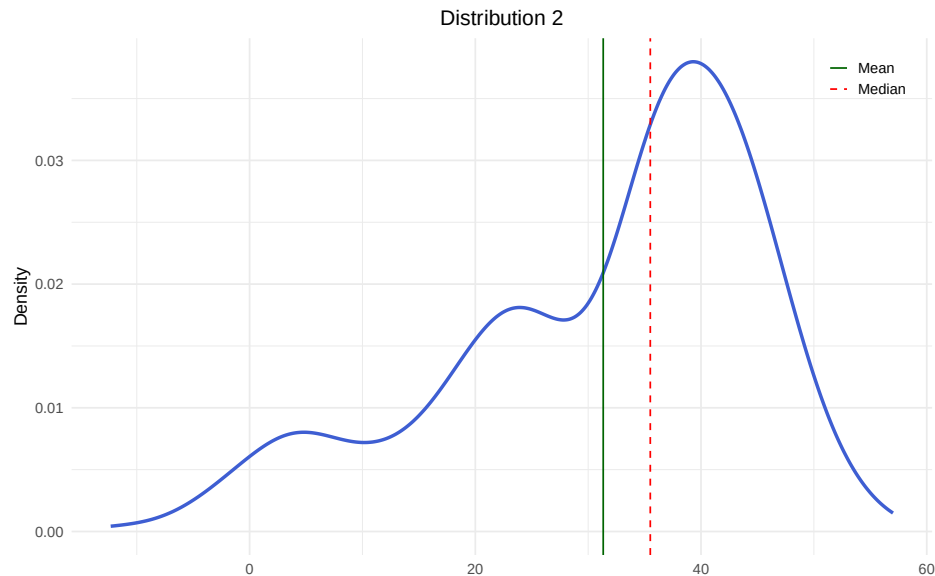
```
base_theme <- theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = c(0.98, 0.98),
  legend.justification = c(1, 1)
)
```

```
base_scales <- list(
  scale_color_manual(values = c("Mean" = "darkgreen", "Median" = "red")),
  scale_linetype_manual(values = c("Mean" = "solid", "Median" = "dashed"))
)
# -----
```

```
line_d123 <- data.frame(
  x = c(mean_d123, median_d123),
  type = c("Mean", "Median")
)
```

```
ggplot(data.frame(d123 = d123), aes(x = d123)) +
  geom_density(color = "#3e5ed2", linewidth = 1) +
  geom_vline(data = line_d123,
    aes(xintercept = x, color = type, linetype = type),
    key_glyph = "path",
    linewidth = 0.5) +
```

```
labs(title = "Distribution 2", x = NULL, y = "Density",
     color = NULL, linetype = NULL) +
base_scales + base_theme
```



(b) Create a “Distribution 3”:

- A single dataset that is normally distributed (bell-shaped, symmetric).
Note: you do not need to combine datasets, just use the `rnorm()` function to create a single large dataset ($n=800$).
- Compute the mean and median.
- Draw lines showing the mean (solid) and median (dashed).

```
nd <- rnorm(n = 800, mean = 100, sd = 12)
```

```
mean_nd <- mean(nd)
mean_nd
```

```
## [1] 100.4225
```

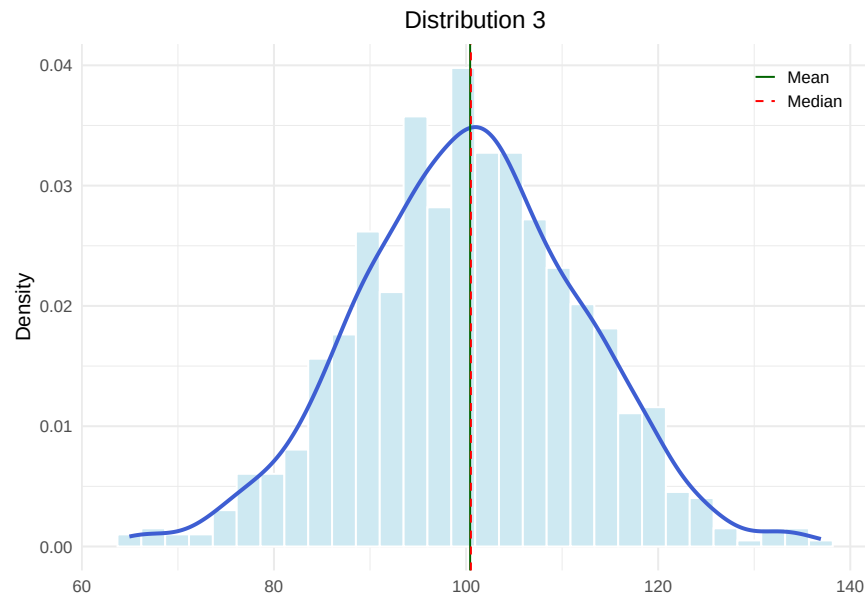
```
median_nd <- median(nd)
median_nd
```

```
## [1] 100.5014
```

```
line_nd <- data.frame(
  x = c(mean_nd, median_nd),
  type = c("Mean", "Median")
)
```

```
ggplot(data.frame(nd = nd), aes(x = nd)) +
  geom_histogram(aes(y = after_stat(density)),
                fill = "#cee9f2", color = "white") +
  geom_density(color = "#3e5ed2", linewidth = 1) +
  geom_vline(data = line_nd,
            aes(xintercept = x, color = type, linetype = type),
```

```
key_glyph = "path",
linewidth = 0.5) +
labs(title = "Distribution 3", x = NULL, y = "Density",
color = NULL, linetype = NULL) +
base_scales + base_theme
```



(c) In general, which measure of central tendency (mean or median) do you think will be more *sensitive* (will change more) to outliers being added to your data?

I think the **mean** is more sensitive to outliers because it uses every data value, so extreme values can pull it strongly. The **median** is determined by the middle value of the sorted data, so adding outliers usually causes smaller changes because they affect the tails.

Question 2

(a) Create a random dataset (call it `rdata`) that is:

- Normally distributed with $n=2000$, $\text{mean}=0$, $\text{sd}=1$.
- Draw a density plot and put a solid vertical line on the mean, and dashed vertical lines at the 1st, 2nd, and 3rd standard deviations to the left and right of the mean.

Note: You should have a total of 7 vertical lines (one solid, six dashed).

```
rdata <- rnorm(n = 2000, mean = 0, sd = 1)

mean_rdata <- mean(rdata)
sd_rdata <- sd(rdata)
sd_set <- c(-3, -2, -1, 1, 2, 3) * sd_rdata
```

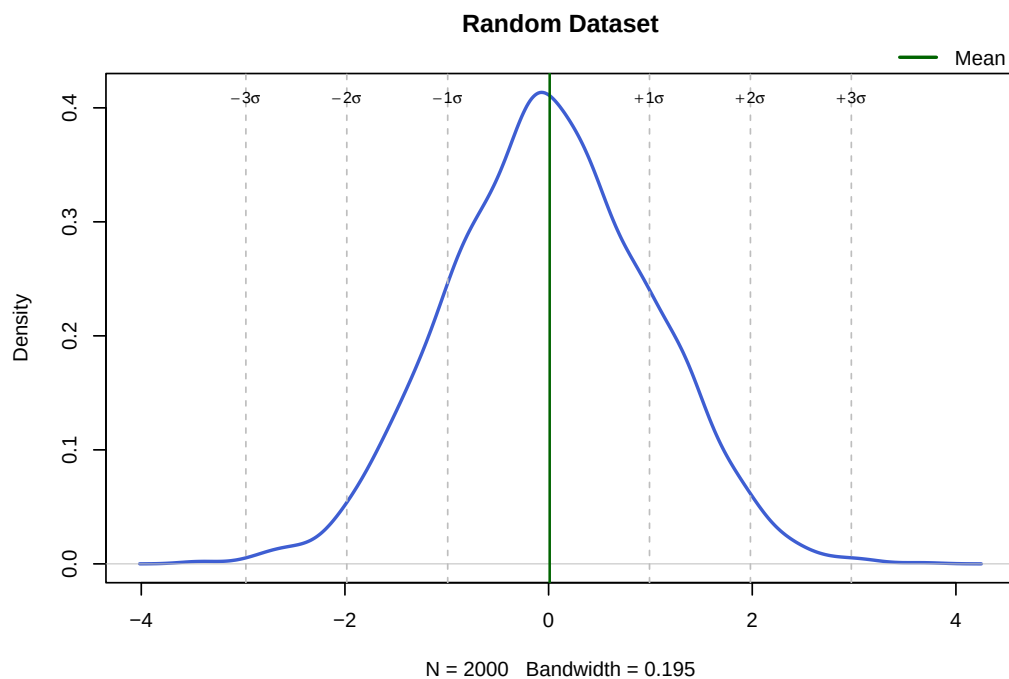
```
plot(density(rdata), col = "#3e5ed2", lwd = 2, main = "Random Dataset")
abline(v = mean_rdata, col = "darkgreen", lwd = 1.5)
abline(v = sd_set, col = "gray", lty = 2)
```

```

y_top <- par("usr")[4] * 0.95
text(
  x = sd_set, y = y_top,
  labels = c(expression(-3 * sigma), expression(-2 * sigma),
    expression(-1 * sigma), expression(+1 * sigma),
    expression(+2 * sigma), expression(+3 * sigma)),
  cex = 0.8
)

legend(
  "topright", inset = c(0, -0.08), legend = "Mean", col = "darkgreen",
  lty = 1, lwd = 2,
  bty = "n", # no box around the legend
  xpd = NA # allow to beyond the plot area
)

```



(b) Using the `quantile()` function,

- Which data points correspond to the 1st, 2nd, and 3rd quartiles (i.e., 25th, 50th, 75th percentiles) of `rdata`?
- How many standard deviations away from the mean (divide by standard-deviation; keep positive or negative sign) are those points corresponding to the 1st, 2nd, and 3rd quartiles?

i. Get the quartiles first

```

quantiles <- quantile(rdata, probs = c(0.25, 0.5, 0.75))
quantiles

##          25%          50%          75%
## -0.672974756 -0.007079331  0.684595879

```

ii. Calculate Z-scores for the quartiles

Note: It represents how many standard deviations are away from the mean. Positive values indicate above the mean, while negative values indicate below the mean.

$$\text{z-score} = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

```
sd_from_mean <- (quantiles - mean_rdata) / sd_rdata
sd_from_mean
##          25%          50%          75%
## -0.68970922 -0.01773379  0.68025681
```

(c) Now create a new random dataset that is normally distributed with: $n=2000$, $\text{mean}=35$, $\text{sd}=3.5$.

- In this distribution, how many standard deviations away from the mean (use positive or negative) are those points corresponding to the 1st and 3rd quartiles?
- Compare your answer to (b).

```
rdata2 <- rnorm(n = 2000, mean = 35, sd = 3.5)

quantiles2 <- quantile(rdata2, probs = c(0.25, 0.75))
quantiles2
##          25%          75%
## 32.59017 37.16169

sd_from_mean2 <- (quantiles2 - mean(rdata2)) / sd(rdata2)
sd_from_mean2
##          25%          75%
## -0.6538743  0.6552393
```

Compared it with Q2(b), we can find that the result of 1st and 3rd quartiles are similar, which are around ± 0.68 standard deviations away from the mean, respectively.

This is because both datasets are normally distributed, and the quartiles of a normal distribution are always located at the same relative positions in terms of standard deviations from the mean.

(d) Finally, recall the dataset **d123** shown in the description of question 1.

- In that distribution, how many standard deviations away from the mean (use positive or negative) are those data points corresponding to the 1st and 3rd quartiles?
- Compare your answer to (b).

```
# d123 from the description of Question 1
d1 <- rnorm(n = 500, mean = 15, sd = 5)
d2 <- rnorm(n = 200, mean = 30, sd = 5)
d3 <- rnorm(n = 100, mean = 45, sd = 5)
d123 <- c(d1, d2, d3)

quantiles_d123 <- quantile(d123, probs = c(0.25, 0.75))
quantiles_d123
##          25%          75%
## 13.45385 30.29733
```

```
sd_from_mean_d123 <- (quantiles_d123 - mean(d123)) / sd(d123)
sd_from_mean_d123
##          25%          75%
## -0.7755917  0.6776238
```

Compared it with Q2(b), we can find that this result of 1st and 3rd quartiles aren't following the same pattern we found previously.

This is because the distribution of `d123` is not normal, and the presence of skewness and multiple modes can affect the relative positions of the quartiles in terms of standard deviations from the mean.

Question 3

Note that, for any dataset d , we can compute the number of bins (k) from the bin width (h):

$$k = \left\lceil \frac{\max(d) - \min(d)}{h} \right\rceil \quad (1)$$

and compute bin width from number of bins:

$$h = \frac{\max(d) - \min(d)}{k} \quad (2)$$

(a) From the question on the forum,

- Which formula does Rob Hyndman's answer (1st answer) suggest to use for bin widths/number?
- Also, what does the Wikipedia article say is the benefit of that formula?

Rob Hyndman's suggest using the **Freedman-Diaconis rule** for bin widths(h), which is represented as Eq. (3):

$$h = 2 \times \frac{\text{IQR}}{\sqrt[3]{n}} \quad (3)$$

where n is the sample size.

Scott's rule is optimal for normally distributed data in the sense that it minimizes the integrated mean squared error of the density estimate. The rule is Eq. (4):

$$h = \frac{3.49 \hat{\sigma}}{\sqrt[3]{n}} \quad (4)$$

Compared with *Scott's rule*, the **Freedman-Diaconis rule** replaces *standard deviation* with **IQR**. Since IQR is based on the middle 50% of the data, it is less affected by extreme values, so the Freedman-Diaconis rule is less sensitive to outliers.

(b) Given a random normal distribution: `rand_data <- rnorm(800, mean=20, sd = 5)`.

```
rand_data <- rnorm(800, mean = 20, sd = 5)
rand_n <- length(rand_data)
rand_range <- max(rand_data) - min(rand_data)
```

- Sturges' formula

We can get the number of bins (k) using the formula:

$$k = \lceil \log_2 n \rceil + 1 \quad (5)$$

```
# Number of bins (k)
sturges_k <- ceiling(log2(rand_n)) + 1
sturges_k

## [1] 11
```

We then use Eq. (2) to calculate the bin width(h).

```
# Bin width (h)
sturges_h <- rand_range / sturges_k
sturges_h

## [1] 3.244189
```

- **Scott's normal reference rule** (uses standard deviation)

We use Eq. (4) to get the bin width first.

```
# Bin width (h)
scott_h <- 3.49 * sd(rand_data) / (rand_n ^ (1 / 3))
scott_h

## [1] 1.890706
```

The number of bins(k) can be derived from the bin width(h) using Eq. (1).

```
# Number of bins (k)
scott_k <- ceiling(rand_range / scott_h)
scott_k

## [1] 19
```

- **Freedman-Diaconis' choice** (uses IQR)

We use Eq. (3) to get the bin width first.

```
# Bin width (h)
fd_h <- 2 * IQR(rand_data) / (rand_n ^ (1 / 3))
fd_h

## [1] 1.463783
```

The number of bins(k) can be derived from the bin width(h) using Eq. (1).

```
# Number of bins (k)
fd_k <- ceiling(rand_range / fd_h)
fd_k

## [1] 25
```

(c) Repeat part (b) but let's extend `rand_data` dataset with some outliers:

- Creating a new dataset `out_data`: `out_data <- c(rand_data, runif(10, min=40, max=60))`.

```
out_data <- c(rand_data, runif(10, min = 40, max = 60))
out_n <- length(out_data)
out_range <- max(out_data) - min(out_data)
```

- From your answers above, in which of the three methods does the bin width (h) change the least when outliers are added (i.e., which is least sensitive to outliers)? (briefly) WHY do you think that is?

– Sturges' formula

```
# Number of bins (k)
out_sturges_k <- ceiling(log2(out_n)) + 1
out_sturges_k

## [1] 11
```

```
# Bin width (h)
out_sturges_h <- out_range / out_sturges_k
out_sturges_h

## [1] 5.092054
```

– Scott's normal reference rule

```
# Bin width (h)
out_scott_h <- 3.49 * sd(out_data) / (out_n ^ (1 / 3))
out_scott_h

## [1] 2.25321
```

```
# Number of bins (k)
out_scott_k <- ceiling(out_range / out_scott_h)
out_scott_k

## [1] 25
```

– Freedman-Diaconis' choice

```
# Bin width (h)
out_fd_h <- 2 * IQR(out_data) / (out_n ^ (1 / 3))
out_fd_h

## [1] 1.46511
```

```
# Number of bins (k)
out_fd_k <- ceiling(out_range / out_fd_h)
out_fd_k

## [1] 39
```

```
orig_h <- c(sturges_h, scott_h, fd_h)
out_h <- c(out_sturges_h, out_scott_h, out_fd_h)

data.frame(
  Method = c("Sturges", "Scott", "Freedman-Diaconis"),
  Original_h = orig_h,
  With_Outliers_h = out_h,
  Change_h = out_h - orig_h
)

##           Method Original_h With_Outliers_h    Change_h
## 1          Sturges   3.244189         5.092054  1.847864923
## 2           Scott   1.890706         2.253210  0.362503751
## 3 Freedman-Diaconis   1.463783         1.465110  0.001327007
```

From the table above, *Freedman-Diaconis' choice* is the least sensitive to outliers for bin width (h). I think this is because:

- (1) Bin width of *Sturges' rule* is driven by sample size and range. Adding only a few outliers changes sample size very little, so the bin count changes only slightly; however, outliers can expand the full range directly and h will increase notably.

- (2) Then compared with *Scott's rule*, it uses IQR instead of standard deviation, so extreme values have a smaller impact on h .